

Fluid: towards transparent, self-explanatory research outputs

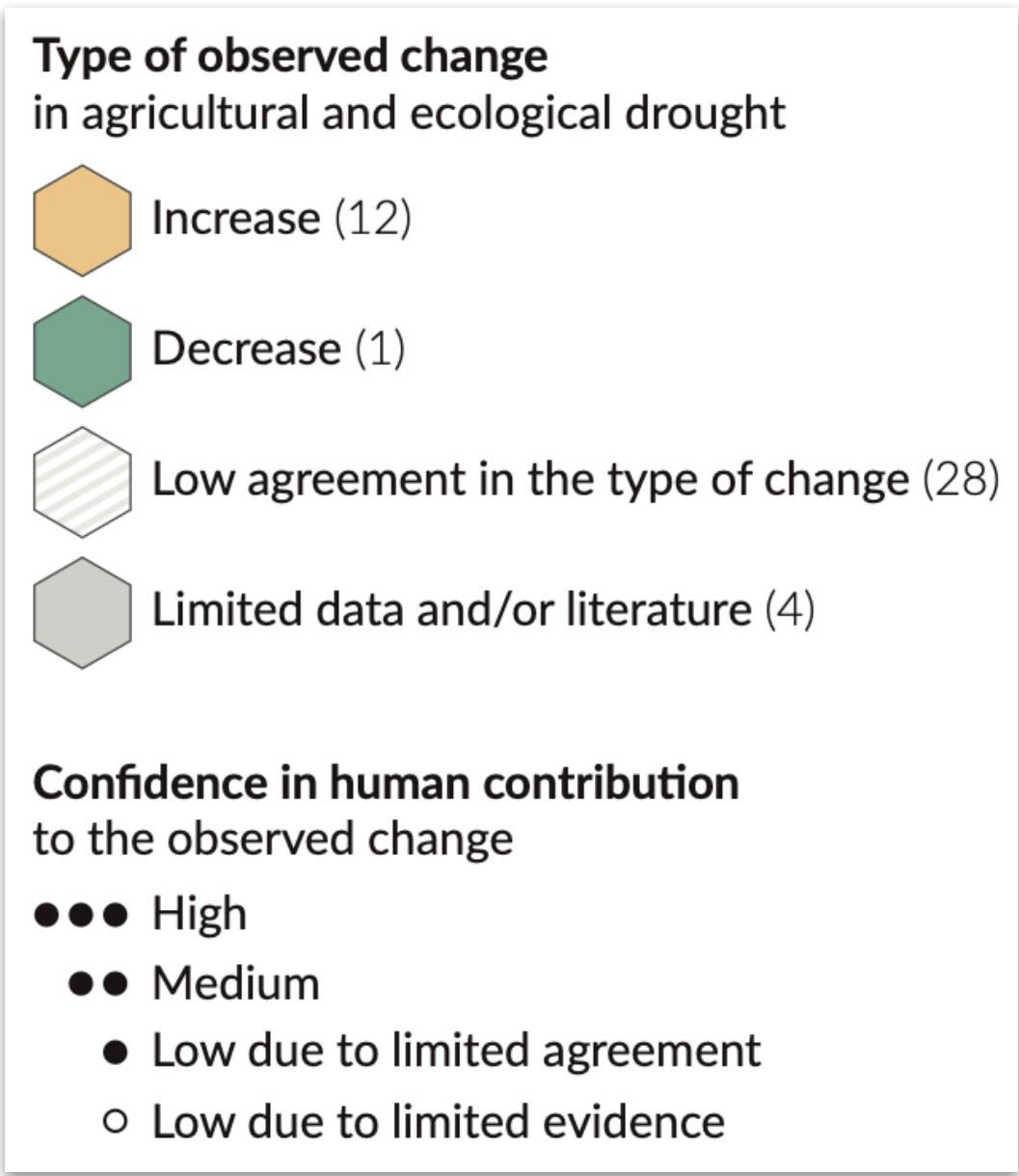
Joe Bond^{*}, Cristina David^{*}, Minh Nguyen^{*}, Roly Perera^{*†}

^{*}University of Bristol

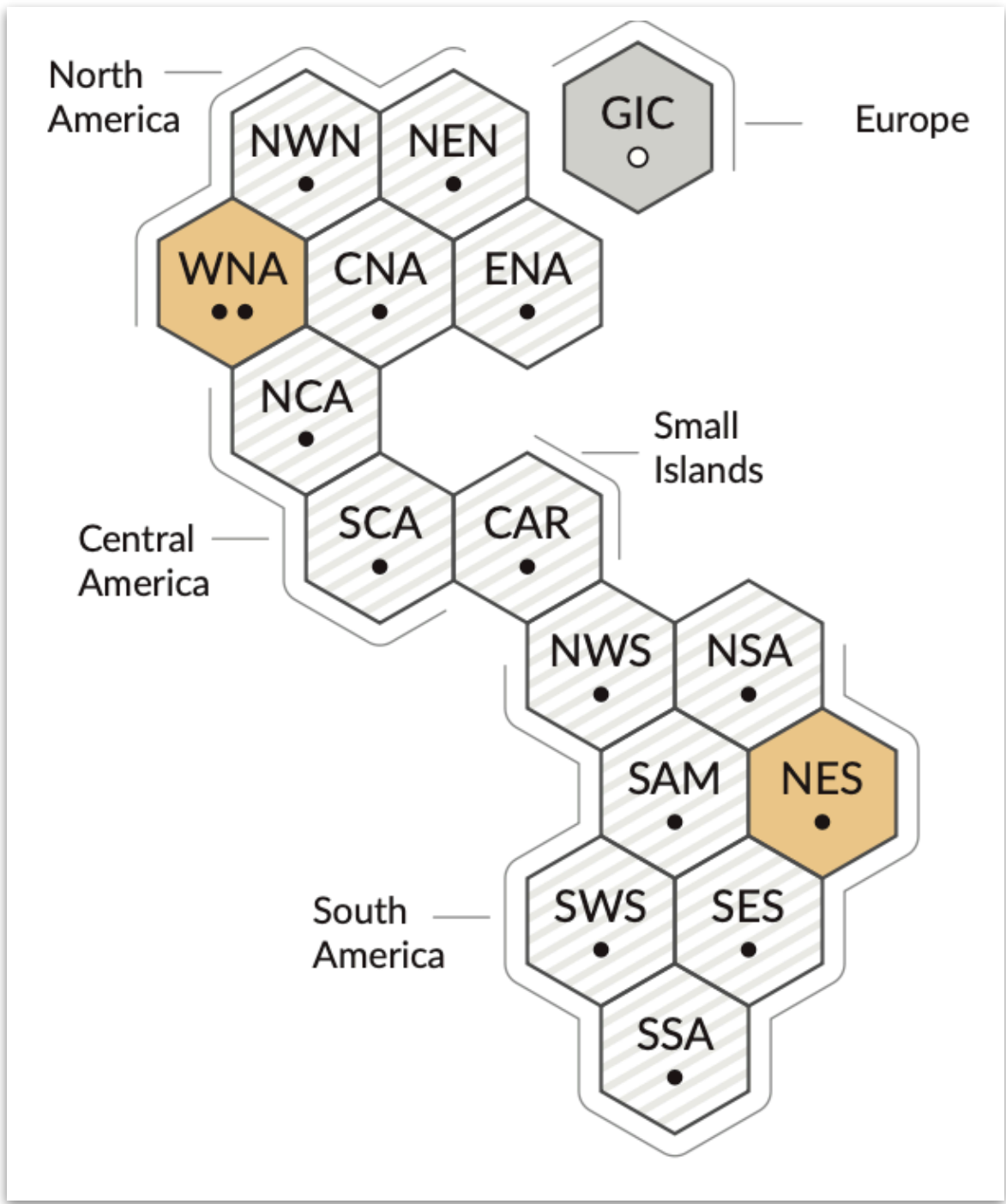
[†]Institute of Computing for Climate Science, University of Cambridge

Understanding, trusting and evaluating visual information

Matters to policy makers, scientists, journalists, educators (and just regular people)



Legends and captions of limited value



Informative in a certain sense but

- what are we looking at?
- where did it come from?
- why should we should trust it?

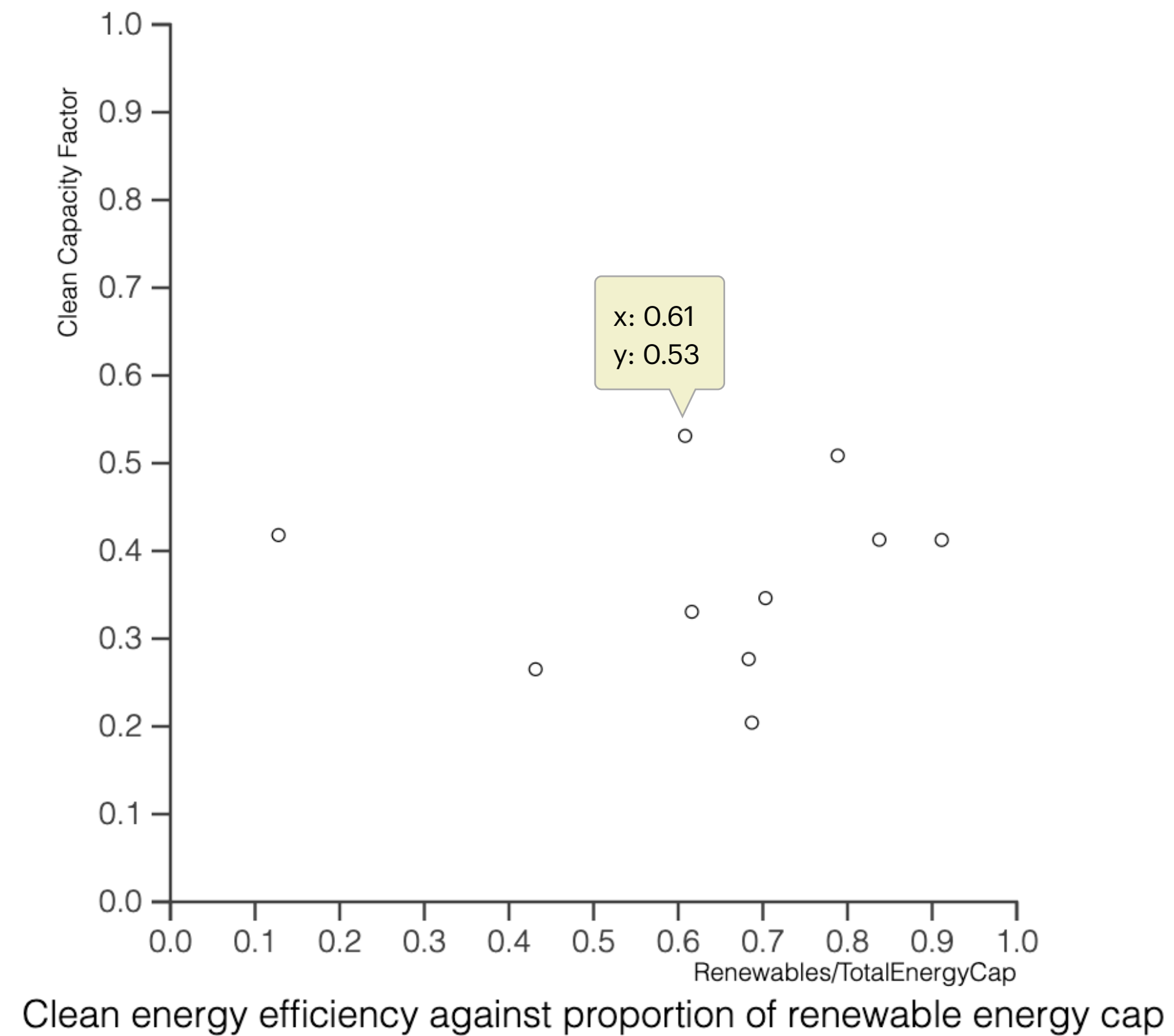
Familiar problem for traditional print media, but what about digital media?

Can PL help?

Heating of the climate system has caused global mean sea level rise through ice loss on land and thermal expansion from ocean warming. Thermal expansion explained 50% of sea level rise during 1971–2018, while ice loss from glaciers contributed 22%, ice sheets 20% and changes in land-water storage 8%. The rate of ice-sheet loss increased by a factor of four between 1992–1999 and 2010–2019. Together, ice-sheet and glacier mass loss were the dominant contributors to

Supporting text references computed data

Towards transparent, self-explanatory data visualisation



Questions we might ask

What year is this data from?

Is each circle a country?

What do x and y represent?

What energy types count as “renewable”?

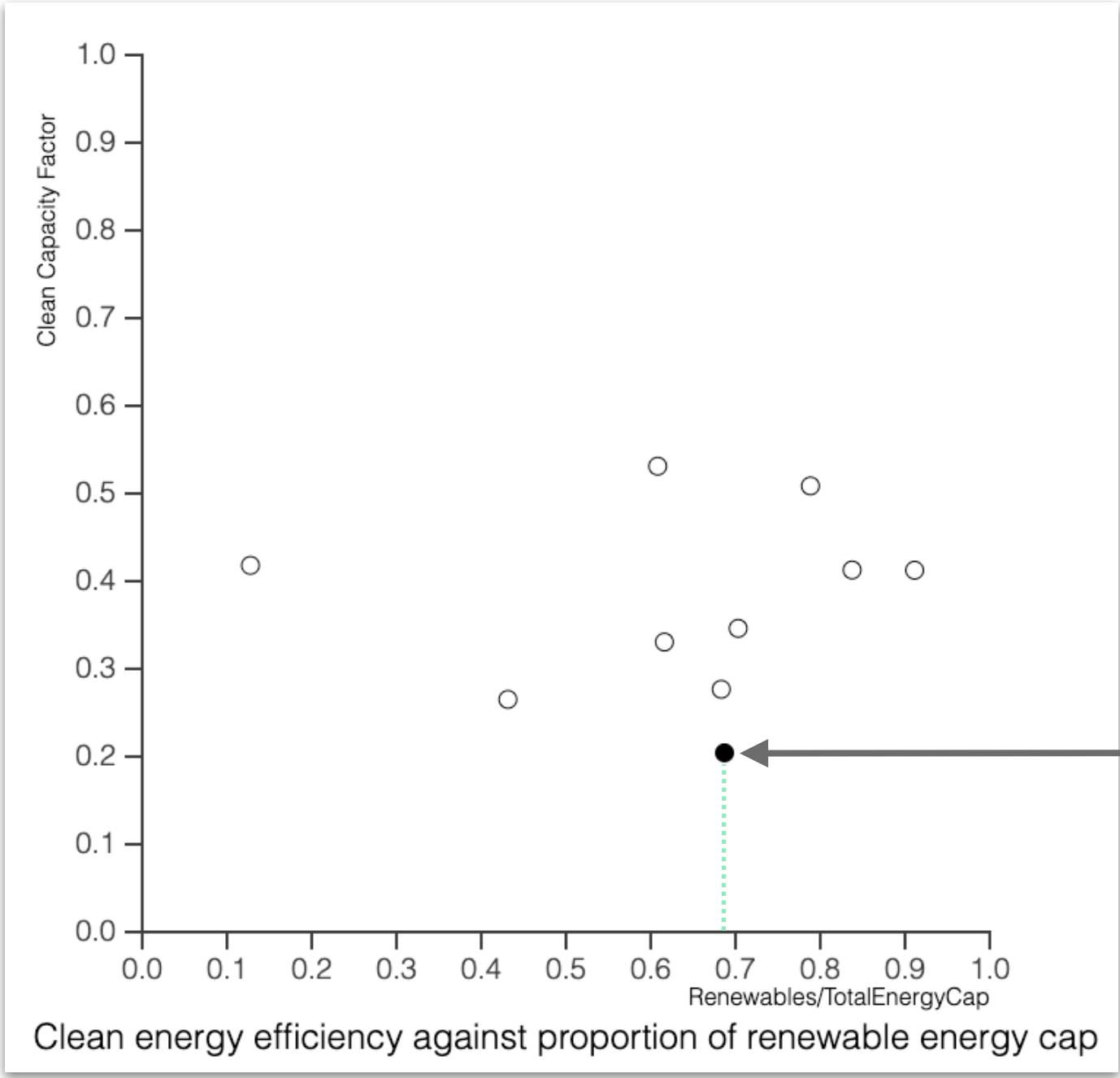
Viz libraries provide tooltips which clarify the **outputs**, but what about the relationship to inputs?

Data and source code help but reverse-engineering relationships is not easy

Demo: related inputs

Demo: related inputs

Related inputs operator $\nabla\Delta$
is not idempotent



Mediating output answers that question and induces a demand on other data — here only the **x-coordinate** is selected ∇

Δ Primary selection asks what demands this data?

non_renewables										
	coalCap	coalGen	country	gasCap	gasGen	nuclearCap	nuclearGen	petrolCap	petrolGen	year
57	405.5	345.13	JPN	747.84	425.37	333.23	49.11	241.69	53.78	2018

renewables					
	capacity	country	energyType	output	year
225	18.22	JPN	Bio	30.13	2018
226	246.51	JPN	Hydro	81.11	2018
227	491.96	JPN	Solar	62.11	2018
228	30.66	JPN	Wind	6.44	2018

non_renewables										
	coalCap	coalGen	country	gasCap	gasGen	nuclearCap	nuclearGen	petrolCap	petrolGen	year
57	405.5	345.13	JPN	747.84	425.37	333.23	49.11	241.69	53.78	2018

renewables					
	capacity	country	energyType	output	year
225	18.22	JPN	Bio	30.13	2018
226	246.51	JPN	Hydro	81.11	2018
227	491.96	JPN	Solar	62.11	2018
228	30.66	JPN	Wind	6.44	2018

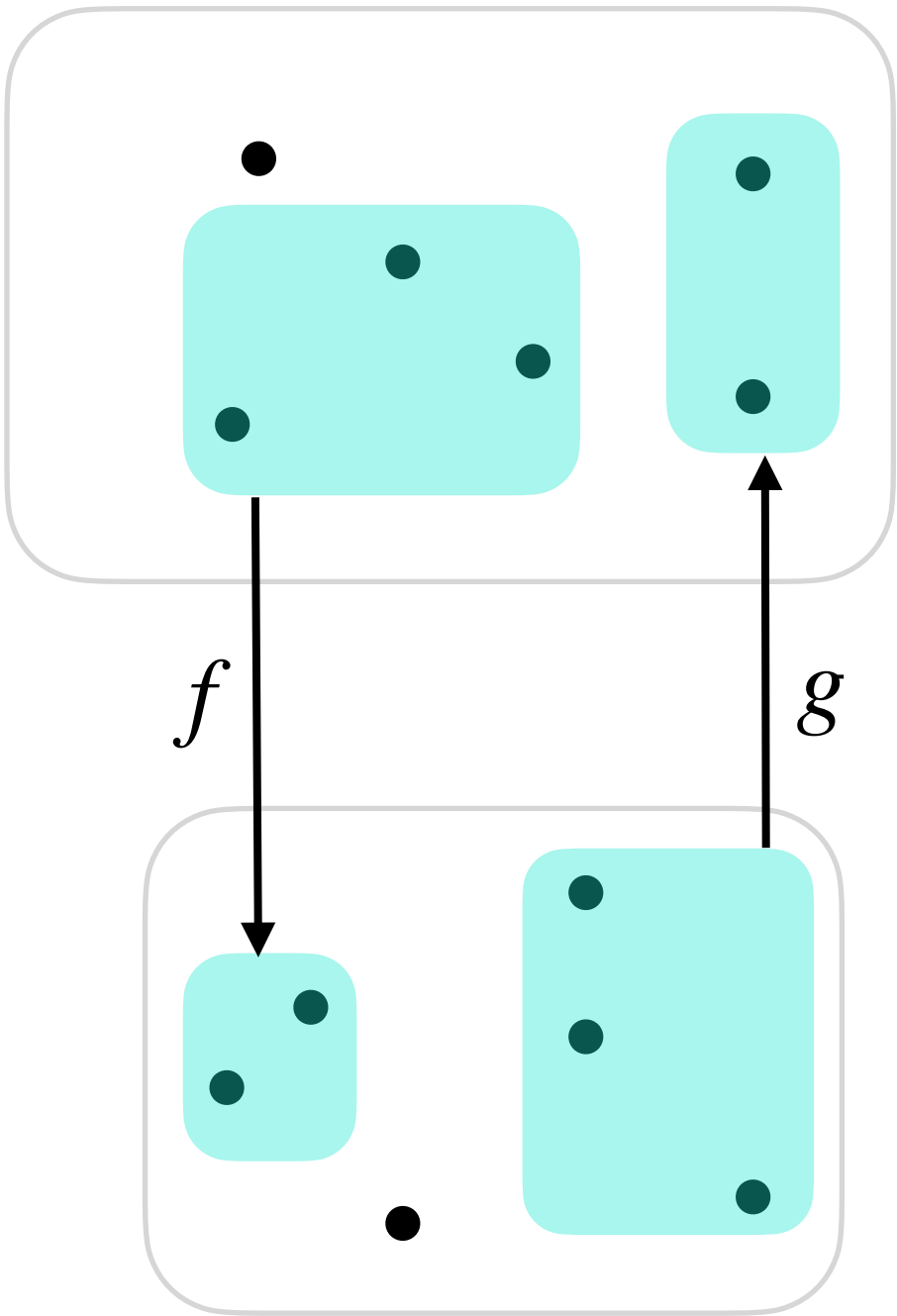
Adding **capacity** for any renewable makes **nuclearGen** and all renewable **output** relevant too — because these are demanded by the **y-coordinate**

Conjugate operators for related inputs

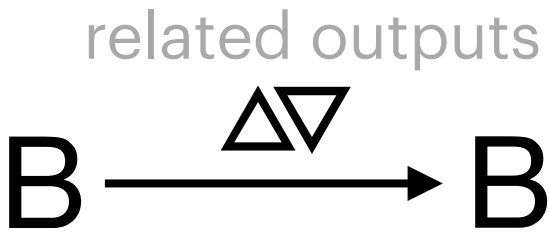
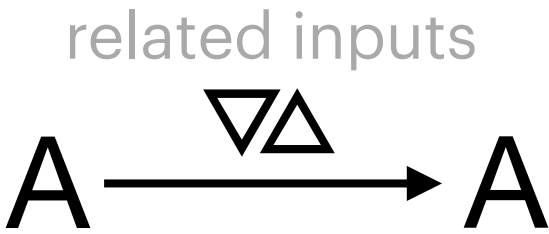
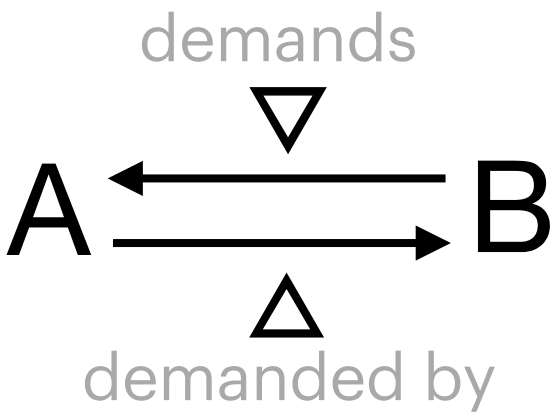
What about
correctness
and
minimality of
 $\triangleright$ and $\triangleleft$?

Diagram
illustrating
conjugacy
(maybe self-
conjugacy)

$$f(x) \wedge y = \perp \iff x \wedge g(y) = \perp$$

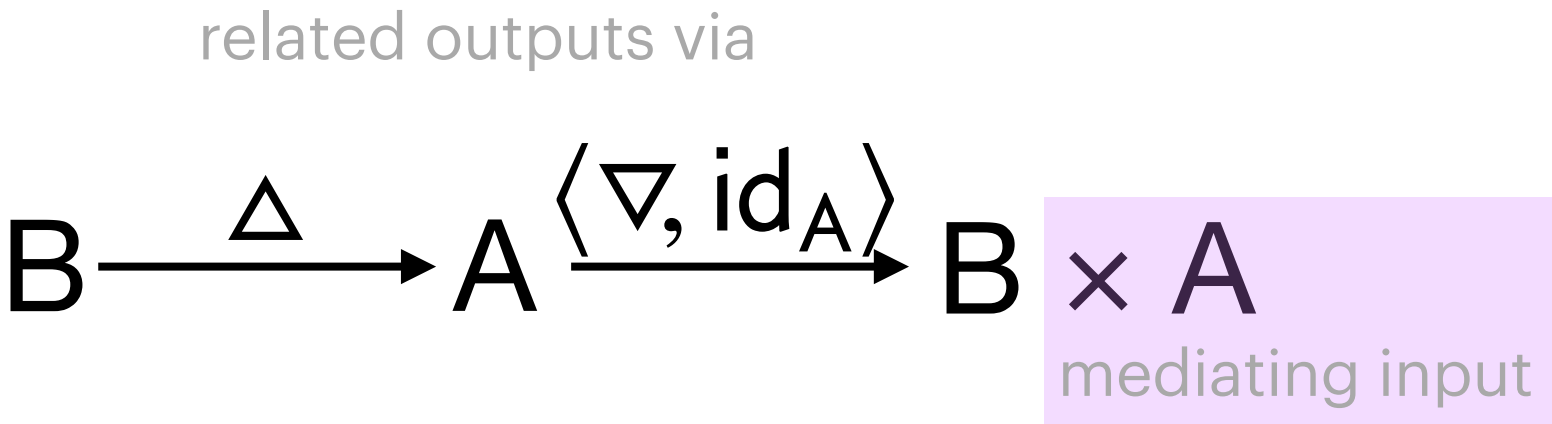
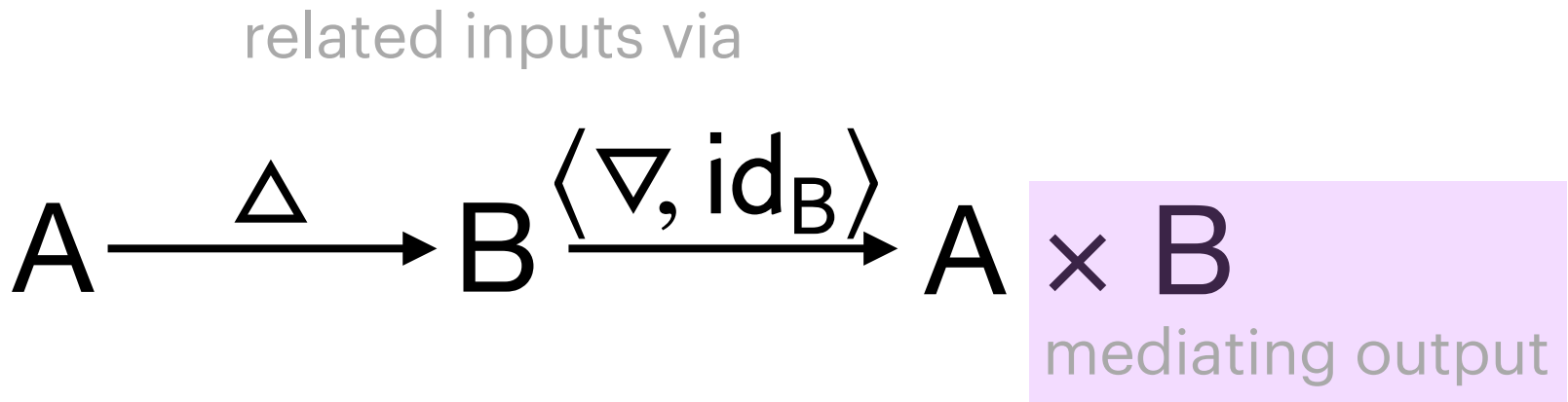


Operators with conjugates admit **products**
Conjugate of $\langle f, g \rangle$ combines two inputs
using either $\wedge$ or $\vee$



conjugate

self-conjugate



```

MultiPlot {
  "bar-chart" :=
    let totalFor c rows =
      sum [ row.output | row ← rows, row.country = c ];
    let data2015 = [ row | row ← renewables, row.year = 2015 ];
    countryData = [ { x: c, y: totalFor c data2015 }
                    | c ← ["China", "USA", "Germany"] ]
    in BarChart {
      caption: "Total output by country",
      data: countryData
    },
  "line-chart" :=
    let series type country = [
      { x: row.year, y: row.output }
      | year ← [2013..2018], row ← renewables,
      row.year = year, row.energyType = type, row.country = country
    ] in LineChart {
      caption: "Output of USA relative to China",
      plots: [
        LinePlot { name: type, data: plot }
        | type ← ["Bio", "Hydro", "Solar", "Wind"],
        let plot = zipWith (fun p1 p2 → { x: p1.x, y: p1.y / p2.y })
                        (series type "USA") (series type "China")
      ]
    }
}

```

Untyped, interpreted functional language
 Records, dictionaries, data types, matrices
 No effects to speak of (only errors)

Program evaluates to result plus **dynamic
 dependence graph** G

Use reachability in G and G^{-1} to implement
 demands and demanded by operators

i front end to visualise inputs, outputs with
 selection information

Related inputs

Dual to **brushing and linking** feature from data visualisation

Not full intensional explanations (future work), but “data transparent”

Related outputs, revisited

Explored dual idea in POPL 2022

Swap one operator by its conjugate to compute related outputs:

$$C \xrightarrow{\nabla_f} A \xleftarrow{\nabla_g} B$$

$$C \xleftarrow{\Delta_f} A \xleftarrow{\nabla_g} B$$

But doesn't lend itself to related inputs:

$$A \xleftarrow{\nabla_f} C \text{ ⚡ } B \xleftarrow{\Delta_g} A$$

More general form is composition of **product view** $\langle f, g \rangle$ with its own conjugate

$$B \times C \xleftarrow{\Delta} A \xleftarrow{\nabla} B \times C$$

with (optional) **projections** to disregard irrelevant views

$$C \xleftarrow{\Delta_{\text{snd}}} B \times C \xleftarrow{\Delta} A \xleftarrow{\nabla} B \times C \xleftarrow{\nabla_{\text{fst}}} B$$

Projection clamps **demand** of discarded part at $\perp$

$$\nabla_{\text{fst}}(b) = (b, \perp) \qquad \Delta_{\text{fst}}(b, c) = b$$

Recovers related outputs but amenable to related inputs

$$A \xleftarrow{\nabla} B \times C \xleftarrow{\Delta} A$$

Focused queries via projection operators

Unfocused query

output					filter	input_image				
5	4	2	5	2		15	13	6	9	16
3	1	2	-1	-2		12	5	15	4	13
3	0	1	0	-1	-2	14	9	20	8	1
2	1	-2	0	0	-1	4	10	3	7	19
1	0	-1	-1	-2	0	3	11	15	2	9
					1					

Related outputs with respect to both inputs

Focused query

output					input_image				
5	4	2	5	2	15	13	6	9	16
3	1	2	-1	-2	12	5	15	4	13
3	0	1	0	-1	14	9	20	8	1
2	1	-2	0	0	4	10	3	7	19
1	0	-1	-1	-2	3	11	15	2	9

Related outputs with respect to input_image only

Composing $\triangleleft$ with projection operator discards irrelevant inputs

Availability of discarded inputs clamped at T

Focused queries via projection operators

Projection operators turn out to be super-useful

Related inputs

compose $\triangleright$ with projection to disregard irrelevant outputs (clamps their demand at $\perp$)

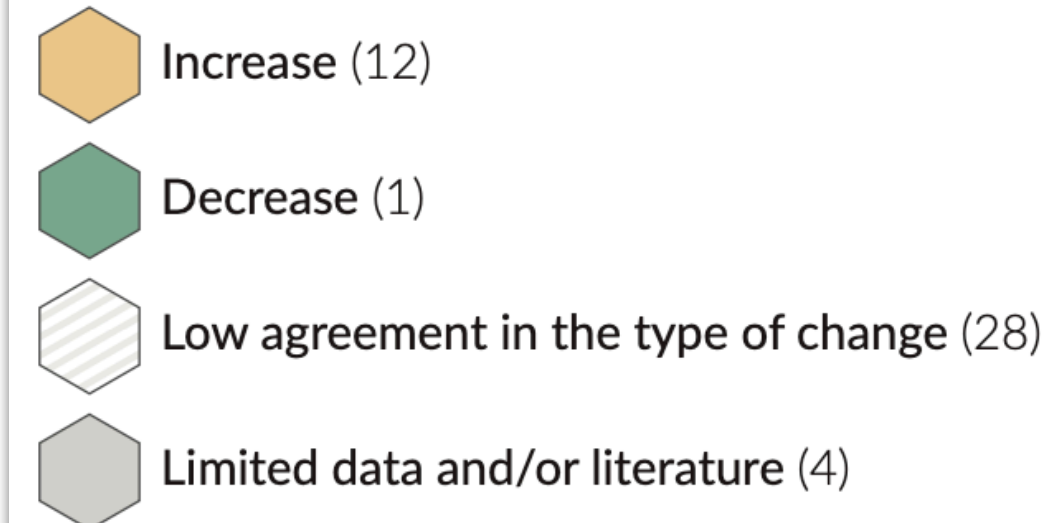
“which inputs are related with respect to these particular outputs” (e.g. x vs y coordinate)

compose $\triangleleft$ with projection to disregard irrelevant sources (clamps their availability at T)

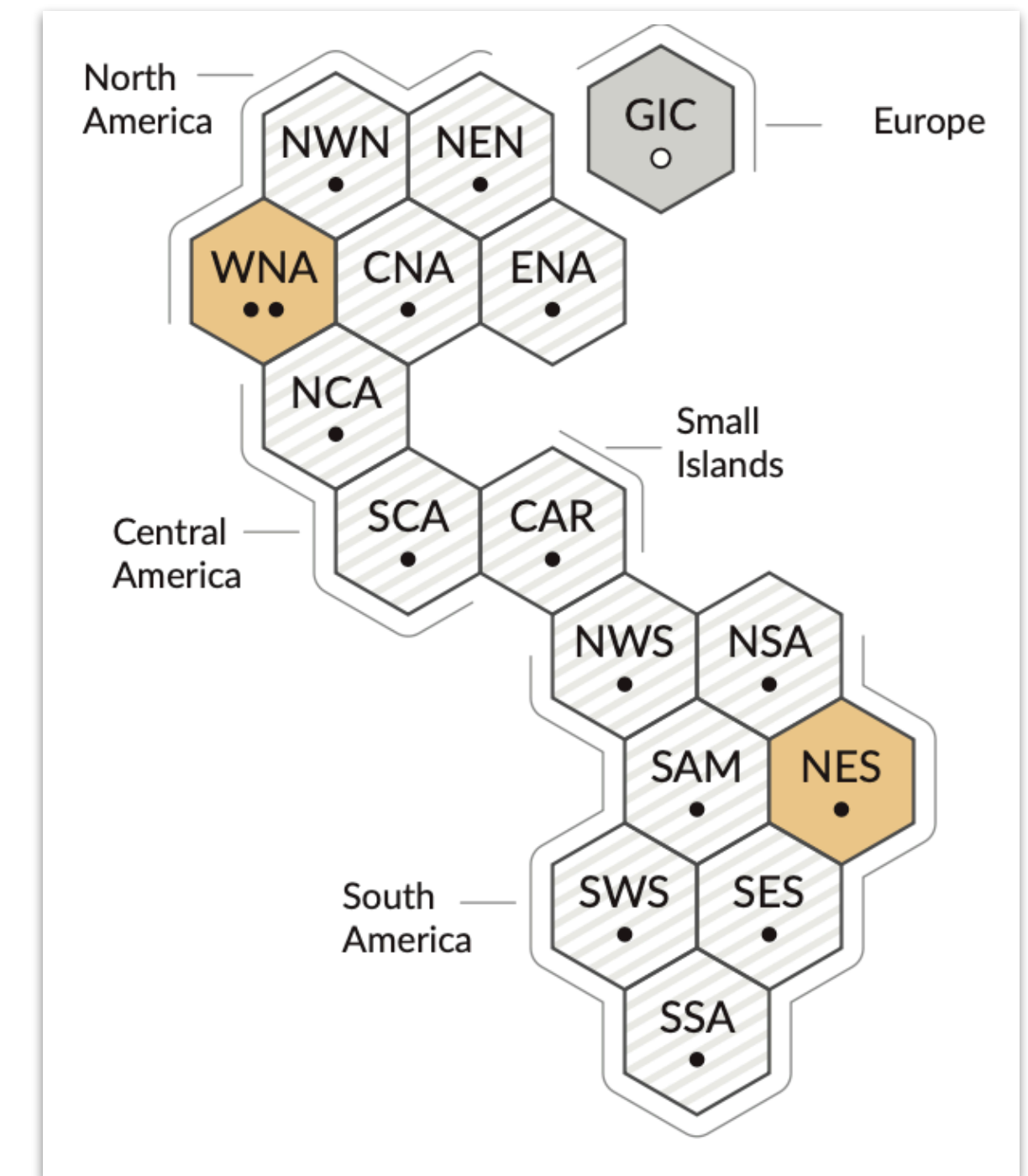
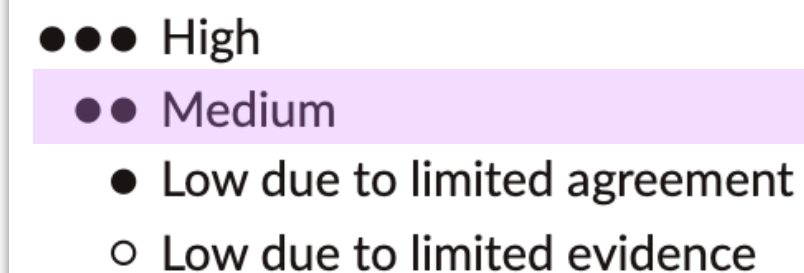
“what of these particular inputs are related?”

Use case

Type of observed change in agricultural and ecological drought



Confidence in human contribution to the observed change



Focus on **layer** of visualisation, e.g. confidence in human contribution, to fix **query context**

What next

Language

Markdown + quasiquotation
Units of measure
Multidimensional arrays
Type system?
Effect system?

Analysis

Intensional explanations
Better dependency relation

<http://f.luid.org>

<https://github.com/explorables-viz/fluid>

Applications

Online articles and long-form essays
presenting environmental science with
transparent and self-explanatory charts and
figures



Institute of
Computing for
Climate Science